

Reverse Engineering Digital ICs through Geometric Embedding of Circuit Graphs

BURCIN CAKIR and SHARAD MALIK, Princeton University, USA

Outsourcing of design and manufacturing processes makes integrated circuits (ICs) vulnerable to adversarial changes and raises concerns about their integrity. Reverse engineering the manufactured netlist helps identify malicious insertions. In this article, we present an automated approach that, given a reference design description with high-level blocks, infers these blocks in an untrusted gate-level (test) implementation. Using the graph connectivity of the netlists, we compute a geometric embedding for each wire in the circuits, which, then, is used to compute a bipartite matching between the nodes of the two designs and identify high-level blocks in the test circuit. Experiments to evaluate the efficacy of the proposed technique on various-sized designs, including the multi-core processor OpenSparc T1, show that it can correctly match over 90% of gates in the test circuit to their corresponding block in the reference model.

CCS Concepts: • **Hardware** → **Partitioning and floorplanning**; • **Security and privacy** → *Hardware reverse engineering*;

Additional Key Words and Phrases: Reverse engineering, partitioning, clustering

ACM Reference format:

Burcin Cakir and Sharad Malik. 2018. Reverse Engineering Digital ICs through Geometric Embedding of Circuit Graphs. *ACM Trans. Des. Autom. Electron. Syst.* 23, 4, Article 50 (July 2018), 19 pages.

<https://doi.org/10.1145/3193121>

1 INTRODUCTION

In a multi-vendor environment where circuits are designed and fabricated in untrusted supply chains, it becomes difficult to detect a malicious change in the design. Reverse engineering to construct the high-level function is an important technique to help expose such changes.

An initial work [19] provides a study of the well-known ISCAS-85 circuits and provides high-level block identification techniques starting from a gate-level netlist for review by a human analyst. These techniques include looking for shared control functions, global wires like clock or reset signals, finding repeated circuit slices and library modules, identifying bus structures from the outputs of repeated modules, and using common names that appear in the netlist.

The methods presented in References [24, 34] focus on automatic techniques for functional matching of a set of library components against a flat gate circuit. In terms of matching techniques, these articles are similar to the previous work in the area on template-based circuit understanding

This work was supported in part by C-FAR, one of the six SRC STARnet Centers, sponsored by MARCO and DARPA.

Authors' addresses: B. Cakir, Princeton University, Department of Electrical Engineering, 41 Olden Street, Princeton, NJ, 08544; email: bcakir@princeton.edu; S. Malik, Princeton University, Department of Electrical Engineering, 41 Olden Street, Princeton, NJ, 08544; email: sharad@princeton.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 ACM 1084-4309/2018/07-ART50 \$15.00

<https://doi.org/10.1145/3193121>

using logic equivalence and model checking methods [15, 25, 27]. There are also some methods that focus on identifying finite-state machines (FSMs) in a given netlist from the feedback loops of the design [32], but these matching techniques are not complete and not capable of matching complex components, leaving large parts of the test circuit unmatched.

Another recent attempt that targets matching parts of the circuit against a set of predefined library components employs a subgraph isomorphism algorithm to find the components within a given subcircuit [33]. Choosing a subset of possible input and output patterns, they construct a graph both for the circuit block to be searched and the candidate components. They do not do a logic analysis but rather suggest a possible matching to be analyzed by an equivalence checker afterwards. The accuracy and the complexity of the method depend on the set of simulation vectors used to construct the graphs and the sizes of the corresponding graphs of the library components and the given subcircuit. As the authors also mention in the article, the initial subcircuit identification still remains a problem to be tackled.

Apart from sub-circuit matching algorithms, there has been work on using clustering-based techniques for automated circuit partitioning [3–6, 10, 11, 14, 17, 18, 29, 31]. The articles on such methods employ different algorithms, mostly graph-based, ranging from random walks to spectral decomposition, to divide the circuit into disjoint clusters. Several placement algorithms have been studied for that purpose. However, these techniques do not provide a high-level block matching algorithm between two different versions of a design.

The main challenge in reverse engineering is finding meaningful component boundaries in a sea of logic gates based on structural information. To the best of our knowledge, none of the previous techniques suggests a global division of the circuit into high level blocks and ours is the first successful high-accuracy approach to identify a large scale partition of a design. Partitioning techniques such as spectral clustering-based algorithms [5, 17] identify circuit partitions but do not address the issue of matching these clusters to behavioral blocks in a reference model. In our work, we study the problem where the initial behavioral or register-transfer level (RTL) description of the circuit is available and we need to find a meaningful functional partition of the design we received from manufacturing (i.e., test circuit) matching the behavioral/RTL block partitions. We attack the problem by following a graph embedding approach. For both circuits, the reference and test, we compute a d^* -dimensional geometric embedding where each wire in the circuits maps to a point in the corresponding d^* -space. After a vector alignment step (where necessary), we use a bipartite matching algorithm to find correspondence between the nodes of the test and reference circuit. The final output of the algorithm outputs a labeling for each wire in the test circuit using the known block partition in the original design. This can then be followed by a verification step where automated techniques possibly aided by a human analyst are used to verify that the individual blocks in the partition of the test circuit match those of the reference circuit. This verification step is beyond the scope of this article. We do not do any logic equivalence check, nor consider gate logic/types in the design for functional verification. Our work presents a novel attack on the reverse engineering problem that is different than previously explored strategies. It helps design engineers automatically infer a higher level representation of an untrusted circuit and presents a complementary tool to the aforementioned techniques. Experimental results on the efficacy of the algorithm show that we can achieve up to 99% matching accuracy, including 92% coverage for OpenSparc T1, a large open-source processor.

The rest of the article is organized as follows. Section 2 provides an overview of the approach and presents the design analysis flow. Section 3 provides the experimental results on different benchmarks. Section 4 provides a discussion of the proposed technique and concluding comments.

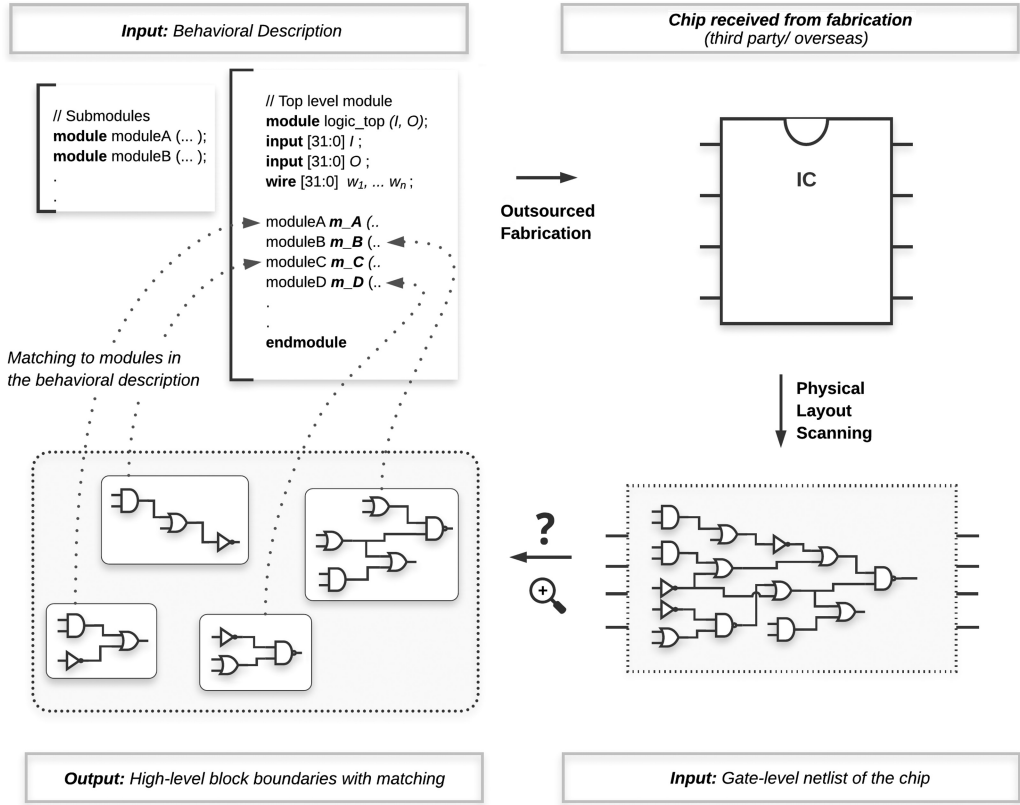


Fig. 1. Although designers may have the initial RTL design, due to the outsourcing of the manufacturing process, they do not have control over the synthesis and fabrication of the chip. Our technique provides a method that, given a reference circuit implementing the original behavior, would help infer high-level blocks in the fabricated test IC.

2 PROPOSED APPROACH

We address the *functional partitioning for reverse engineering problem* where we want a high-level block partition of the IC that has been received from manufacturing. Figure 1 illustrates an overview of our solution. We assume that the original design is available in some hardware description language (HDL), as a behavioral or RTL source code, that can be synthesized to obtain a gate-level circuit, referred to as the *reference circuit*. We also assume that we have the untrusted manufactured design, referred to as the test circuit, also in a gate netlist and ready to be analyzed. Identifying high-level blocks in the test circuit corresponding to the reference design can help expose malicious changes inserted during design and production flow. Our approach presents a novel technique that leverages very useful information that is already available to the designer: the high-level blocks in the reference HDL design. Functional partitioning for reverse engineering seeks to label gates in the test design with high-level blocks in the reference HDL that they likely are part of. This labeling provides a partition of the test circuit for which each partition can be separately analyzed and even verified against the high-level block that it corresponds to. The focus of this article is on the partitioning and labeling. The analysis/verification can be done using known techniques [34] and is beyond the scope of this article. Our work should be married with

a fine matching technique for it to serve as a detection tool for malicious insertions. (In our threat model, we do not make any assumption about the CAD tool. Since a design engineer can choose different CAD tools available for design synthesis and check the results with different netlists obtained from each such tool against each other, our algorithm provides a robust way to identify high-level blocks with comparable outputs. If the designer is not sure about the trustability of a CAD tool, then s/he may use different tools together with the one in hand to obtain different reference graphs and check each output to verify the rough boundaries outlined in the test circuit from different reference graphs. We wanted to note our suggestions on this matter for readers who will be interested in using our algorithm, yet do not have access to a trustable synthesis tool.)

Note that an attacker may still insert malicious circuits inside blocks. This motivates the need for further work to detect malicious insertions within the blocks identified by our matching technique. Our matching algorithm is still useful in that it helps identify smaller blocks that may be further analyzed.

The proposed approach uses the graph similarity of two functionally similar designs, i.e., the reference and test circuits, and outputs a correspondence between the two sets. The steps of our algorithm are summarized as follows:

Step 1: This requires preparing a gate-level circuit for the reference design by synthesizing the design description to obtain a flat netlist if necessary. [reference netlist]

Step 2: Then, the corresponding circuit graphs are obtained using the flat netlists of both the reference and test circuits. [circuit graphs]

Step 3: Next, a geometric embedding is computed for all nodes in the graphs. This gives a mathematical representation that provides us a medium where we can compare the nodes of the two circuits in a low-dimensional space. [geometric embedding]

Step 4: The embedding coordinates are then aligned to compute a bipartite assignment between the gates of the two netlists for establishing node correspondence between the two circuits. [coordinate alignment]

Step 5: The gates in the reference circuit are labeled with the high-level block they belong to. In the next step, a bipartite matching is computed. The labels from the reference design are used to determine the corresponding functional units in the test circuit. [bipartite matching/node labeling]

Step 6: Since the results of the assignment might be noisy, we include a final clean-up stage. Then, a high-level representation of the test design is provided as output. [clean-up]

A. Embedding and Feature Extraction (Steps [1–4])

For the embedding stage, we suggest two alternative algorithms to compute the features of the nodes, which can afterwards be merged together into a richer set before the final matching: eigendecomposition and primary input/output-based features.

1.1. Eigendecomposition-Based Features (Steps [1–3]). The first embedding approach is based on the eigendecomposition of the circuit graphs. Thus, it requires getting the graph representation of the netlists. If the reference design was provided as HDL source code, then it is synthesized to obtain a flat gate-level netlist. Therefore, both the reference and the test circuits are assumed to be in gate-level after this stage. Figure 2 illustrates an example logic wiring with its corresponding circuit graph, $G_x(V, E)$, where each wire is represented by a vertex in V and an edge from E indicates the input-output relation of a logic component [9]. An undirected graph is preferred, since we want to capture the functional relation between the nodes regardless of their direction. Then, the edges

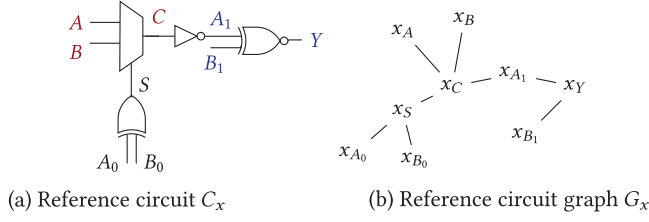


Fig. 2. The reference circuit on the left is obtained by synthesizing the reference HDL. The reference graph on the right is obtained from the reference circuit.

	A_0	S	B_0	A	C	B	A_1	Y	B_1
A_0	•	1	•						
S	1	•	1						
B_0	•	1	•						
A				•	1	•			
C				1	•	1	1		
B				•	1	•			
A_1							1	•	•
Y							1	•	1
B_1							•	1	•

v_{e_1}
1
1
1

v_{e_2}

1
1
1

v_{e_3}

1
1
1

Fig. 3. Affinity matrix, A , where the dots and 1's show areas where the corresponding edge weights are high when $k = 3$ for the circuit in Figure 2. v_{e_1} , v_{e_2} , and v_{e_3} are example eigenvectors that represent the dominant components of the decomposition.

in the graph are assigned a similarity weight based on their connectivity. Let $\Gamma(x_p)$ denote the neighborhood of the node x_p with its all adjacent vertices, such that $\Gamma(x_p) = \{x_t \in V | \{x_p, x_t\} \in E\}$, the similarity $s(x_p, x_q)$ between nodes x_p and x_q is defined as follows:

$$s(x_p, x_q) = \frac{1}{\sqrt{|\Gamma(x_p)|} \sqrt{|\Gamma(x_q)|}}. \quad (1)$$

This normalization step checks the degree of each node and adjusts the weights on the edges to reflect the graph similarity among the neighbors. It helps us reduce the weights on the edges connecting the hubs and outliers in the graph by increasing the denominator for the nodes that serve as a bridge among different clusters (due to the high vertex degree of the node) and the ones that are weakly connected to a dense part of the graph (due to the high vertex degree of the neighbors).

Next, an affinity matrix is constructed based on the adjacency list and the k -nearest neighbors $\Gamma_k(x_p)$ of each vertex. While computing the closest neighbors, the distance between each vertex x_p and its adjacent vertices $x_q \in \Gamma(x_p)$ is defined to be the inverse of the corresponding similarity value, $s(x_p, x_q)$. The distance between non-adjacent vertices x_p and x_t , s.t. $x_t \notin \Gamma(x_p)$ but x_p is connected to x_t through multiple edges, $x_p \rightsquigarrow x_t$, is calculated as the sum of the distances on the shortest path that connects them. The distance of a node to itself is defined as 0. Once the neighbor computation is done, the sets are edited with possible additions to achieve a symmetric matrix. Figure 3 illustrates an affinity matrix for the example circuit in Figure 2 constructed by setting $k = 3$. The values in the matrix represent the proximities of the nodes, i.e., the inverse of the corresponding path distance. The entries that are shown with 1 denote the actual adjacency relation whereas the dots show the k closest neighbors with high weights.

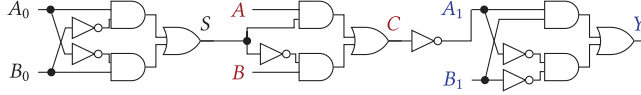


Fig. 4. Logic implementation representing an example netlist for the test circuit C_y .

The affinity matrix helps us define the functional groups in the logic. The clusters in the matrix are defined by the affinity of nodes (wires) in the graph; not the absolute paths between the wires, but their connectedness. Therefore, in the next step, we need a method that will help identify these high correlation regions regardless of the order of nodes in the matrix. Eigendecomposition provides a mathematical framework to factorize a matrix into a canonical form such that it can be represented in terms of its eigenpairs (λ_e, v_e) that satisfy the following equation:

$$A v_e = \lambda_e v_e, \quad (2)$$

where A denotes the affinity matrix and v_e represents the eigenvector with its corresponding eigenvalue λ_e . If the affinity matrix is symmetric, which holds in our case due to the undirected edge construction, then the eigenvectors form an orthonormal basis [22], so that each row (or column) can be represented in terms of its eigenvectors as their linear combination: $\sum_i \alpha_i v_{e_i}$. From Figure 3, we see that the expected eigenvectors that will have high coefficients in the linear combination approximating the rows of the matrix are likely to be in the form of the vectors provided on the right, where again the regions emphasized with 1's are of high value compared to the rest. The rows corresponding to the nodes of each of the three clusters in the matrix can be constructed by taking one of those eigenvectors as the basis, and using small coefficients on the other vectors to adjust the sum in the equation according to the exact values in the row. Therefore, the nodes in one cluster, can be encoded using the values in the corresponding row of the eigenvectors. For example, the node Y in the circuit, considering just the dominant eigenvectors and ignoring the rest, can be coded as $\langle 0, 0, 1 \rangle$, which gives us an embedding with a relatively small dimension ($d = 3$) compared to matrix size.

Overall, the main idea behind techniques using matrix decomposition is based on the insight above. However, there is a bit more math involved in constructing the initial matrices and determining the number of eigenvectors for the embedding. In our method, we use the NJW-algorithm, suggested by Ng et al. [28] to construct a normalized Laplacian from our affinity matrix and obtain the features. Since our matrix is symmetric and only a certain number of vectors are needed, the computation we require is a special one, called the truncated symmetric eigenvalue eigendecomposition, where the factorization is performed to get only the first smallest (or largest) d orthogonal eigenvectors. The complexity of the computation depends on the number of rows in the square matrix together with the estimate for the dimension of the embedding (d) and it can be really fast for sparse matrices [16, 23]. The dominant eigenvectors are identified by checking the eigenvalues of the decomposition. We provide a discussion on how to determine those eigenvectors and decide on the number of them at the end of Section 3 while explaining our experimental results.

Let the logic gates in Figure 2 represent our reference design, and we have a test circuit synthesized using a low-level tech-library in Figure 4. The test circuit implements the same function with more primitive gates and has the circuit graph, G_y , provided in Figure 5. Looking at this circuit graph, we can see that, if we compute the node affinities for this new circuit, the corresponding matrix will have the same skeleton as the one in Figure 3 and the dominant eigenvectors will be in similar shapes compared to those of the reference design, yielding a comparable embedding between the nodes of the two circuits. Therefore, although the circuits do not exactly have the same graph, the similarity between them is seen in the factorization of the matrices.

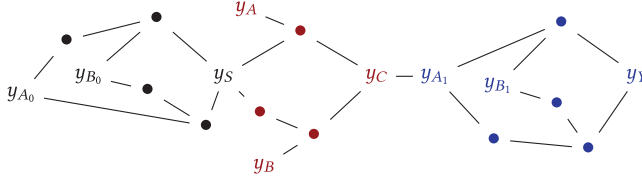


Fig. 5. Test graph G_y , where dots represent the unlabeled intermediate wires in Figure 4.

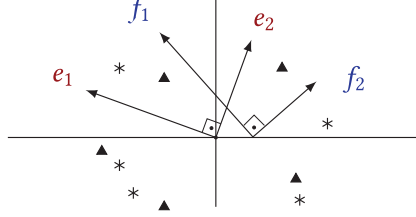


Fig. 6. A small two-dimensional example where triangle and asterisk dots represent the nodes from different circuits while e_1, e_2 and f_1, f_2 are imaginary projection axes to help visualize eigenvector embedding.

At first blush, eigenvectors can be seen as an alternative similar to ratio-cut partitioning techniques to distinguish the local clusters in a given graph. However, they provide much better results than local search approaches and present a global view of the graph without getting trapped in local minima for large-scale optimization problems. They are preferable for our purpose where we want to have a comparable coordinate system between two circuits rather than a connectivity-based clustering that, based on the connectivity of the nodes, may output clusters with different granularity than what we expect. They enable us to present a guided partitioning where we label the clusters according to our reference circuit and calculate a matching using the coordinates without being concerned about the hierarchy of the clusters in the circuit graph, and hence, have control on the number and shape of the partitions.

In our algorithm, we use the orthogonal eigenvectors in each embedding to approximate a linear span for a d -dimensional space (d -space) $d \leq d_1$ and $d \leq d_2$, where d_1 and d_2 represent the number of vectors in the decomposition of the reference and test circuit, respectively. Our approach does not require the dimensions of the initial embedding, d_1 and d_2 , to be equal. The corresponding eigenvectors v_{e_k} , $1 \leq k \leq d_1$ and v_{f_l} , $1 \leq l \leq d_2$ may also have different shapes $m \times 1$ and $n \times 1$ depending on the sizes of reference and test circuits. Hence, we need a preparation step to adjust the embedding coordinates for comparison. To do that, first, subtract the mean of each eigenvector from itself so that the values on the coordinate axes can be centered around the origin on both embeddings. Then, we follow a more formal adjustment step to find a correspondence between the axes. The details of the technique are provided next.

1.2. Feature Alignment (Step 4). Figure 6 shows a two-dimensional example where triangle dots denote the nodes of the first circuit while asterisk ones represent the other set. e_1, e_2 and f_1, f_2 are axes of the two embeddings, respectively. In our approach, we do not actually do an axis computation, what we obtain from the feature embedding step rather is a projection of points onto those axes. For example, for the first circuit, the eigenvectors, $\vec{v}_{e_1}$ and $\vec{v}_{e_2}$, consist of values that hold the position of the nodes relative to e_1 and e_2 . The axes below are imaginary components that help us understand the values on the eigenvectors.

The eigenvectors that we calculate for each graph give two separate vector sets that span the same coordinate space. From the assumption that two circuits have similar functionality, we

postulate that they approximately cover the same space. However, that does not necessarily imply that they are aligned and there is a correspondence for each axis between the two representations. Although we try to center the eigenvectors around zero by subtracting the mean of each from itself, due to the noisy data the centroid of the two data sets may not align. Each embedding, as mentioned, can also have a different number of vectors with different sizes, $m \times 1$ and $n \times 1$, determined by the number of nodes in each graph. Even when the sizes are equal, an eigenvector in one set can be some linear combination of a few others from the second one. However, to have a comparable geometric encoding, need a deformation step to make sure that the embedding axes are centered around the same mean, aligned with the same angles and scaled proportionately. Without any further information other than the given two vector sets, normally this problem does not have an easy solution.

What we additionally know is the information on the embedding coordinates of some nodes that have a fixed matching correspondence: the Input/Output (I/O) pins of the design. By our similarity assumption of the circuits, we know that the eigenvectors in each embedding represent two different span sets of our d -dimensional space, i.e., by a linear combination of vectors in one set we can construct (approximately in our case, due to the limited eigenvector selection) any vector within the d -space. Using these two pieces of information, we come up with the following approach for feature adjustment.

For each set, we first form a new matrix such that the rows in the matrices represent the I/O pins and the columns consist of the values coming from the eigenvectors, i.e., the embedding coordinates of these nodes. Hence, if there are r different I/O pins, we construct an $r \times d_1$ and $r \times d_2$ matrix with the same I/O row ordering. The columns in the matrices are a subset of the eigenvectors from each embedding and have different values. However, from our assumption above, we know that a column in one matrix can be also represented as a linear combination of the columns of the second one. For each column $\vec{v}_k$ in the first matrix, we thus compute a set of coefficients $\lambda_{k,i}$ to multiply the columns of the second matrix $\vec{u}_i$, $i = 1, 2, \dots, d_2$, and construct an approximation $\vec{v}_k^*$ for the vector in the first set.

$$\vec{v}_k^* = \sum_{i=1}^{d_2} \lambda_{k,i} \vec{u}_i, \quad k = 1, 2, \dots, d_1. \quad (3)$$

Hence, this is a well-formulated optimization problem to be solved; a multivariate linear regression problem with r samples (size of the training set) and d_2 variables. For the row indexed t , we have the tuple $\{\vec{v}_{k,t} : \vec{u}_{1,t}, \vec{u}_{2,t}, \dots, \vec{u}_{d_2,t}\}_{t=1}^r$ where the first term being the regressands (outputs) and the rest the regressors (inputs) of the optimization.

After the coefficients are calculated, we can compute a correspondence for each $\vec{v}_k$ in the first system, and align the values on each axis (column) of the second matrix. To adjust the values of all nodes on the original embedding, we follow the exact procedure described as above. But, now we use the calculated coefficients on the eigenvectors itself, rather than the I/O pins, and replace the $\vec{u}_i$ in Equation (3) with its corresponding eigenvector. The axes are shifted, rotated and made parallel to each other. And, finally, we normalize each row to norm 1 to repeat the final step in the NJW-algorithm referred to above [28]. For each $\vec{v}_{e_k}$ of the first set we now have a corresponding eigenvector $\vec{v}_{e_k}^*$ in the second embedding.

The above calculations suggest a way to find an aligned feature set of dimension d_1 for the test design. Thus, the final size of the embedding d is set to d_1 and the eigenvector dominance is measured according to eigenvalues of the first embedding, since the vector alignment is done by taking it as the alignment basis. (We prefer to choose the reference circuit as the first set so that

the alignment basis is done according to the original design. However, the reverse also yields a similar approximation.)

The two coordinate systems coming from the eigendecomposition are represented with $Y = [\vec{v}_{e_1}^*, \vec{v}_{e_2}^*, \dots, \vec{v}_{e_d}^*]_{n \times d}$ and $X = [\vec{v}_{e_1}, \vec{v}_{e_2}, \dots, \vec{v}_{e_d}]_{m \times d}$. The former is the embedding for our test circuit and the latter for the reference design. Each row in this representation holds the embedding coordinates (features) of the nodes. For an element in the i th row of Y and one in the j th row of X , the coordinates are written as follows: $\vec{y}_i = (Y_{i,1}, Y_{i,2}, \dots, Y_{i,d})$ and $\vec{x}_j = (X_{j,1}, X_{j,2}, \dots, X_{j,d})$. The number of nodes in each circuit can be different, hence, the sizes of the test and the reference design are denoted by n and m , respectively.

2. Primary Input/Output-Based Features (Steps [1–3], [4]). Apart from the spectral embedding of the circuit graphs, another useful information that we can exploit is the basic graph connectivity in each design, i.e., the distances between graph vertices. To take this into account, for each primary I/O port of the graph, we compute its shortest distance to every other node to obtain a metric that tells the relative location of nodes with respect to the I/O ports of the circuit. We do the computations in the same row order as X (or Y) so that we can append them as new feature columns to X (or Y) later if needed. The distance between vertices is computed according to the definition provided in the previous section for k -nearest-neighbor calculations, however, on a directed graph. The previously given normalization formula is replaced by the following similarity metric:

$$s_{p \rightarrow q}(x_p, x_q) = \frac{1}{\sqrt{|\Gamma_{out}(x_p)|} \sqrt{|\Gamma_{in}(x_q)|}}, \quad (4)$$

where $s_{p \rightarrow q}$ denotes the normalized weight of a directed edge from node x_p to x_q . $\Gamma_{out}(x_p)$ and $\Gamma_{in}(x_q)$ are the sum of the edges outgoing from x_p and the sum of the edges incoming to x_q , respectively. Though, we preferred to obtain eigenfeatures based on a symmetric graph due to its robustness, here, when computing directed IO features we include the information coming from the logic flow and the direction of the gate connections.

Then, following the same argument explained for eigenfeatures, each row is normalized to have a unit length. No alignment between the features of the two circuits is needed for I/O-based embedding.

We now have a comparable representation of the nodes and are ready for the matching stage.

B. Matching and Final Labeling (Steps [5 and 6])

For the final matching stage, we either use the eigenfeatures alone or choose the feature embeddings calculated based on the shortest distances with respect to the primary input/output ports instead, or concatenate the two of them and do the matching using all the features we have—in a higher dimension. In either case, the final *embedding vector* for node y_i is denoted with $\vec{y}_i^*$ and similarly, node x_j is represented by $\vec{x}_j^*$. We provide experimental results for all three.

Using a method parallel to the one in Reference [7], next, we compute an assignment matrix G to hold the similarities between the two node sets. For each y_i and x_j pair, a similarity measure is calculated according to the following Gaussian form:

$$g_{ij} = \exp(-d_{ij}^2/(2\sigma^2)), \quad (5)$$

where d_{ij} denotes the Euclidean distance between $\vec{y}_i^*$ and $\vec{x}_j^*$ and σ is the standard deviation of all d_{ij} values. The result of the equation gives an approximation for the geometric proximities of the nodes according to their d -dimensional embeddings. Similar to pre-normalization steps applied to data for general machine-learning techniques, this step helps construct a graph with normalized weights for better accuracy. According to this equation, the edges between two nodes that have

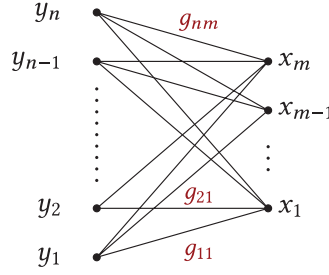


Fig. 7. Bipartite matching between the nodes of the two embeddings.

vastly different feature vectors, hence large d_{ij} values, are assigned smaller similarity weights. And the ones that are very close to each other (the nodes that have very similar features) get assigned values close to 1. Hence, we obtain a normalized set of weights for similarity. Now, what we need to solve is a maximum weight matching between two bipartite sets.

Figure 7 shows the corresponding bipartite graph, $G(Y, X; E)$, between the nodes of the two embeddings. In our assignment matrix, the rows of the matrix correspond to nodes of the set Y and the columns represent the elements of X . The entries are the weights of the edges between the nodes in the figure. Since the number of elements in each set is different, the size of our matrix is $n \times m$. However, to have a balanced and complete graph, we can assume that the assignment matrix is square, by adding dummy nodes with zero-weight edges where necessary and obtain a new $\tilde{n} \times \tilde{n}$ matrix. The reason why we want our matrix to be square is to make it suitable for a range of different algorithms proposed for similar assignment problems, which can be employed in the next step.

The dual of the maximum weight matching problem is called minimum cost matching, and has a well-known polynomial time solution with the Hungarian algorithm for balanced bipartite graphs, where $|X| = |Y| = \tilde{n}$. The method is a combinatorial optimization algorithm that, given an $\tilde{n} \times \tilde{n}$ cost matrix, allows a minimum matching to be found in $O(\tilde{n}^3)$ complexity [13]. In practice, the runtime of the algorithm never approaches this asymptote. In our tool, we preferred to employ a more recent solution proposed by Volgenant [35] for linear assignment problems that can run orders of magnitude faster on sparse matrices. (A further discussion on reducing the density of the matching graphs for sparse representation is provided at the end of this section.)

In the next step, we convert our similarity matrix to a cost representation C , where each element is obtained by subtracting its value from the largest number in G as follows: $c_{ij} = \max_{i,j}(g_{ij}) - g_{ij}$. The rows of the matrix represent the test circuit and the columns denote the reference design. Then, we run the matching algorithm on C . The algorithm outputs an assignment where each node in the smaller design is assigned to a node from the reference set. If the number of nodes, n , in the test circuit is less than the number nodes, m , in the reference circuit, then we just ignore the dummy nodes in the test set. However, if n is greater than m , then it means that some of the nodes in the test circuit are assigned to dummy nodes in the reference set and, hence, do not have a real match. We will find them a group later in the final clean-up stage.

As mentioned, in the reference circuit, which functional block a node belongs to is assumed to be known. This is true as synthesis tools typically retain HDL module names as prefixes on the wire names during the synthesis of the reference circuit from the reference HDL. Hence, after the matching algorithm, we can obtain a high-level representation of the test circuit using the information coming from the labeling of reference circuit nodes. The nodes are grouped into different clusters in the test circuit based on the label of the nodes they are matched to. For each

functional block in the original design, we have a set of candidate nodes in the test circuit that corresponds to that specific functional unit.

The matching results can sometimes be noisy and certain assignments may not be correct. Due to the size difference of netlists, some nodes may not have been assigned a label as well. For circuits where there is a large difference in the number of nodes between the reference and test circuits, we expect noise and if the test circuit is larger, observe regions where unmatched nodes predominate. Therefore, we include a clean-up stage. Using the connectivity of the test circuit, we first check the labels assigned to the k -nearest neighbors of all nodes, and determine a new label based on the majority and keep the record of each update until the first iteration is done; then change all the labels at once. We believe looking at a wider neighborhood helps us better estimate the actual boundary. In the second and later iterations, we check only the nodes that are not assigned a label yet—unmatched nodes mentioned before. To avoid propagating wrong labels, for smaller circuits a tolerable number of unmatched nodes, we skip the initial correction step, and check only those with no label. The algorithm converges quickly as after each iteration we assign labels to only a few of the remaining nodes. (The nodes that are assigned to intermediate wires connecting the blocks in the reference design are labeled as part of the intermediate logic between the blocks and contribute to the label propagation. Result evaluation considers both block and intermediate logic labels.) An explanation about how to decide on the value of k and how to pick between the clean-up methods suggested for large and smaller circuits is provided in the following section.

Once the labeled clusters have been obtained for the test circuit, we have effectively partitioned the netlist into high-level modules. This addresses the major problem of determining block boundaries from a sea of gates in a given test netlist. The identified blocks in the test circuit can be then be analyzed or even verified against the corresponding blocks in the reference design [34], this step is beyond the scope of this article.

Discussion on reducing the runtime complexity of the matching algorithm: When dealing with large designs, to achieve a better runtime, we convert our original full bipartite graph (see Figure 7) into a sparse representation first. We decrease the number of edges in the graph by picking a parameter ρ_s to include only a certain number of edges in the rows and columns of the similarity matrix, and ignore the rest. In each row and column of the original matrix, we mark the ρ_s entries with the highest values and use only those entries to construct the new sparse graph. In this way, we get rid of the edges that connect the nodes that have a low probability to be matched in the assignment algorithm, while keeping the most relevant ones. After that simplification, both the memory consumption and the complexity of the assignment algorithm decreases. Then, in the matching part, we use an implementation based on the package provided in Reference [21] for the Jonker-Volgenant algorithm for dense and sparse matrices [20, 35].

3 EXPERIMENTAL RESULTS

We tested our approach on eight different designs from OpenCores [1] and the open source multi-core processor OpenSparc T1 [2] on a server with Intel® Xeon® E5-2680 v4 CPU clocked at 2.40GHz running Linux 7.4 (requesting a maximum of 64GB of RAM and 24 threads). To have different implementations of each design, we used the Cadence Encounter RTL Compiler (RC) and Synopsys Design Compiler (DC) to synthesize them with (1) IBM/ARM cell library for the 45nm SOI process (*soi12*) and (2) the Synopsys 90nm Generic Library (*s90nm*). The HDL compilers are called Tool A (TA) and Tool B (TB) and the technology libraries are referred to as *lib1* (used for reference circuit) and *lib2* (used for test circuit) in the article. Due to academic licensing terms, which one (TA or TB / *lib1* or *lib2*) corresponds to which tool/library (RC or DC / *soi12* or *s90nm*) is kept anonymous. The circuits were also synthesized under different optimization targets. The reference circuit was synthesized for performance (timing) using TA, while the test design was optimized

Table 1. Results for Design Benchmarks I (Reference - Test/time) [TA/lib1 - TA/lib2]

Design Information		Parameters							Matching Results (%)						Notes
Primary ports		#Total wires							Embedding		I/O Features		Eigen Features		I/O & Eigen F.
PI	PO	Refr.	Test	k	ρ_s	#FB	d_1	d_2	[M ₁]	[M ₂]	[M ₁]	[M ₂]	[M ₁]	[M ₂]	
115	152	5141	5483	100	1000	8	7	6	0.83	0.86	0.76	0.79	0.81	0.84	Wishbone memory contr.
74	106	15840	16432	130	4000	12	7	7	0.98	0.98	0.86	0.86	0.90	0.90	MIPS32 processor
53	66	2730	2740	50	500	2	4	4	0.97	0.97	0.88	0.88	0.94	0.94	PID controller
139	128	2965	3203	50	500	16	16	16	0.90	0.95	0.86	0.93	0.90	0.96	Reed-Solomon encoder
119	94	11601	13874	120	5000	9	9	9	0.71	0.85	0.66	0.74	0.65	0.72	8051 microcontroller
1130	1416	22925	22720	200	4000	25	18	18	0.88	0.89	0.77	0.77	0.93	0.93	Wishbone conmax IP
70	40	24160	12916	150	4000	7	18	19	0.96	0.96	0.85	0.85	0.93	0.93	Floating-point unit
259	129	8006	9658	100	500	23	20	20	0.92	0.95	0.91	0.93	0.90	0.93	AES cipher
193	137	96394	126021	350	4000	9	8	7	0.16	0.32	0.47	0.71	0.47	0.72	OpenSpart T1 processor [†]

[†] The initial matching results for sparc benchmark is low is due to the size difference between the two circuits and, hence, the unmatched 30,000 nodes. This is also the reason why, for this design, as explained at the end of Section 2B, we preferred to do an initial clean-up even for the nodes that already have a label.

Table 2. Results for Design Benchmarks II (Reference - Test/area) [TA/lib1 - TA/lib2]

Design Information		Parameters							Matching Results(%)						Notes
Primary ports		#Total wires							Embedding		I/O Features		Eigen Features		I/O & Eigen F.
PI	PO	Refr.	Test	k	ρ_s	#FB	d_1	d_2	[M ₁]	[M ₂]	[M ₁]	[M ₂]	[M ₁]	[M ₂]	
115	152	5141	5221	100	1000	8	7	6	0.85	0.89	0.76	0.78	0.82	0.85	Wishbone memory contr.
74	106	15840	14534	130	4000	12	7	8	0.97	0.97	0.85	0.85	0.90	0.90	MIPS32 processor
53	66	2730	2571	50	500	2	4	4	0.97	0.97	0.87	0.87	0.92	0.92	PID controller
139	128	2965	3165	50	500	16	16	16	0.91	0.96	0.87	0.93	0.91	0.96	Reed-Solomon encoder
119	94	11601	13107	120	5000	9	9	9	0.70	0.83	0.66	0.76	0.67	0.75	8051 microcontroller
1130	1416	22925	22217	200	4000	25	18	18	0.89	0.89	0.78	0.78	0.93	0.93	Wishbone conmax IP
70	40	24160	11891	150	4000	7	18	20	0.96	0.96	0.87	0.87	0.93	0.93	Floating-point unit
259	129	8006	8367	100	500	23	20	20	0.93	0.95	0.92	0.93	0.92	0.94	AES cipher
193	137	96394	91948	350	4000	9	8	10	0.87	0.92	0.73	0.83	0.76	0.85	OpenSpart T1 processor [†]

[†] For this design, as explained in Section 2B, we preferred to do an initial clean-up that includes the nodes that have an initial label as well.

separately with timing and area constraints using both TA and TB to have four different implementations. To show the efficacy of the algorithm on the matching of circuits optimized under different objectives with different CAD tools, we present results for all four combinations: Reference[TA/lib1] - Test/time[TA/lib2], Reference[TA/lib1] - Test/area[TA/lib2], Reference[TA/lib1] - Test/time[TB/lib2] and Reference[TA/lib1] - Test/area[TB/lib2]. For comparison, the reader should bear in mind that it does not matter which circuit, reference or test, is area-optimized and which one is time-optimized, as in our experiments we arbitrarily decide to call one as the reference and other as the test. As will be discussed later, what affects the results most is the difference in the graph sizes and connectivity.

For the matrix calculations and optimization problems, we used Armadillo and mlpack, C++ linear algebra and machine learning libraries [12, 30]. Tables 1, 2, 3, and 4 present a summary of our results on various circuits. The first section gives information about the number of I/O pins and the sizes of each design pair. The section indicated with Refr. represents our reference model synthesized with lib1 cell library by Tool A (TA) while Test represents the test design synthesized using lib2 cell library for area or timing optimization by either Tool A (TA) or Tool B (TB). The data show that the circuit graphs of the reference and test designs are significantly different. For example, the time-optimized OpenSpart test design in Table 1 has about 30,000 more cells than the reference design. While constructing the circuit graphs, buffer structures—including repeating inverter pairs—are removed from the graph representation to help get rid of a lot of additional

Table 3. Results for Design Benchmarks I (Reference - Test/time) [TA/lib1 - TB/lib2]

Design Information		Parameters				Matching Results(%)								Notes	
Primary ports		#Total wires				Embedding				I/O Features		Eigen Features		I/O & Eigen F.	
PI	PO	Refr.	Test	k	ρ_s	#FB	d_1	d_2	[M ₁]	[M ₂]	[M ₁]	[M ₂]	[M ₁]	[M ₂]	
115	152	5141	4392	100	1000	8	7	6	0.88	0.88	0.82	0.82	0.88	0.88	Wishbone memory contr.
74	106	15840	16580	130	4000	12	7	8	0.87	0.87	0.57	0.57	0.61	0.61	MIPS32 processor
53	66	2730	2901	50	500	2	4	4	0.94	0.94	0.86	0.86	0.93	0.93	PID controller
139	128	2965	2881	50	500	16	16	16	0.97	0.97	0.97	0.97	0.99	0.99	Reed-Solomon encoder
119	94	11601	21894	120	5000	9	9	9	0.78	0.85	0.77	0.83	0.77	0.84	8051 microcontroller
1130	1416	22925	17101	200	4000	25	18	18	0.95	0.95	0.69	0.69	0.94	0.94	Wishbone conmax IP
70	40	24160	12892	120	4000	7	18	17	0.89	0.89	0.72	0.72	0.81	0.81	Floating-point unit
259	129	8006	11818	100	500	23	20	20	0.55	0.84	0.53	0.82	0.54	0.84	AES cipher‡
193	137	96394	104444	300	4000	9	8	9	0.75	0.90	0.54	0.62	0.55	0.64	OpenSpart T1 processor†

† For this design, as explained in Section 2B, we preferred to do an initial clean-up that includes the nodes that have an initial label as well.

‡ The initial matching results for aes benchmark is low is due to the size difference between the two circuits and, hence, the unmatched 3,200 nodes.

Table 4. Results for Design Benchmarks II (Reference - Test/area) [TA/lib1 - TB/lib2]

Design Information		Parameters				Matching Results(%)								Notes	
Primary ports		#Total wires				Embedding				I/O Features		Eigen Features		I/O & Eigen F.	
PI	PO	Refr.	Test	k	ρ_s	#FB	d_1	d_2	[M ₁]	[M ₂]	[M ₁]	[M ₂]	[M ₁]	[M ₂]	
115	152	5141	4229	100	1000	8	7	6	0.87	0.87	0.82	0.82	0.90	0.90	Wishbone memory contr.
74	106	15840	16555	130	4000	12	7	8	0.87	0.87	0.51	0.51	0.61	0.61	MIPS32 processor
53	66	2730	2571	50	500	2	4	5	0.94	0.94	0.85	0.85	0.92	0.92	PID controller
139	128	2965	2078	50	500	16	16	16	0.90	0.90	0.93	0.93	0.95	0.95	Reed-Solomon encoder
119	94	11601	12198	120	5000	9	9	8	0.90	0.90	0.79	0.79	0.81	0.81	8051 microcontroller
1130	1416	22925	16731	200	4000	25	18	18	0.95	0.95	0.67	0.67	0.94	0.94	Wishbone conmax IP
70	40	24160	8933	150	4000	7	18	17	0.85	0.85	0.65	0.65	0.76	0.76	Floating-point unit
259	129	8006	10953	100	500	23	20	20	0.47	0.79	0.47	0.77	0.48	0.78	AES cipher‡
193	137	96394	88183	300	4000	9	8	9	0.75	0.89	0.56	0.65	0.57	0.65	OpenSpart T1 processor†

† For this design, as explained in Section 2B, we preferred to do an initial clean-up that includes the nodes that have an initial label as well.

‡ The initial matching results for aes benchmark is low is due to the size difference between the two circuits and, hence, the unmatched 3,000 nodes.

cells and changes inserted during aggressive time optimization stages. In addition to buffers, global wires like clock and reset wires and constants are also ignored to avoid noise in decomposition.

k and ρ_s parameters in the table provide the values used to determine the k -nearest neighborhood and the sparsity of the final assignment matrix fed into the matching algorithm. #FB shows the number of different functional blocks in the original circuits.

In a similar way we obtained the labels for the reference circuit (reading the prefixes on the wires retained from the HDL module names during the synthesis of the reference HDL), in fact, for our experiment, we also have the information about which functional block a node in the test circuit actually belongs to, and use that information to check the accuracy of our final matching. This information about the test netlist is used only to evaluate the final matching results of our tool and nowhere else in the algorithm. [M1] and [M2] provide the results before and after the clean-up stage.

The ρ_s parameter shows the number of entries we decided to keep in each column and row of the cost matrix. It does not have a significant effect on the final matching result but helps us make the bipartite graph sparse and reduce the runtime, for large circuits. When picking a parameter for a design, however, one should be careful about keeping enough entries so that the correct/optimal assignment pairs are active in the cost matrix. A good number can be obtained by setting ρ_s to a value around the average cluster size in the reference circuit. This usually gives a good estimate

Table 5. Percentage of New Elements Added to Make the Affinity Matrix Symmetric

Circuits	TA/lib1	TA/lib2		TB/lib2	
	Reference	Test (Time)	Test (Area)	Test (Time)	Test (Area)
	new elm. (%)	new elm. (%)	new elm. (%)	new elm. (%)	new elm. (%)
Wishbone memory contr.	0.21	0.21	0.21	0.20	0.20
MIPS32 processor	0.18	0.18	0.17	0.20	0.19
PID controller	0.17	0.18	0.18	0.18	0.16
Reed-Solomon encoder	0.10	0.10	0.10	0.12	0.12
8051 microcontroller	0.20	0.19	0.22	0.17	0.16
Floating-point unit	0.13	0.13	0.13	0.13	0.13
AES cipher	0.18	0.17	0.17	0.13	0.13
OpenSpart T1 processor	0.23	0.28	0.24	0.24	0.24

to reduce the density of the initial full matrix without failing the matching algorithm, but one can adjust the parameter according to time or memory limitations or a specific design that needs more samples.

The choice of k changes the affinity matrix and, hence, affects the eigendecomposition. The parameter should be chosen such that it is high enough to remove noise but small enough to keep the clusters separate. A basic approach used commonly is to set $k = \sqrt{n}$, where n is the number of nodes in the design. In our experiment, we did not see a significant influence of k on the matching results, therefore did not include a separate plot for that. The reason for this, we believe, is the high enough number of interconnections between the gates of each functional block in most circuits. (Except the final clean-up stage where we check the k -nearest neighbors of the nodes, by the nature of the method used to compute the shortest paths in the graph, k has no effect on I/O-based features.) As explained before, after k -nearest neighbor calculations are done, the matrix is edited with new additions to achieve symmetry, and hence, an undirected edge construction. Table 5 shows the percentage of new elements added to the affinity matrix after k -nearest-neighbor calculations. Here, the reader should remember that those elements are added to affinity matrix with certain weights. A higher k or a large number of new nodes does not necessarily imply a more strong connection between the nodes of the graph. As explained previously, the edge similarity values are used to create these connections, and the nodes that are distant to each other are connected only weakly.

Deciding the number of eigenvectors is critical for the quality of the embedding. We use the eigengap heuristic [26], where the eigenvectors with non-zero eigenvalues are divided based on the largest gap in the corresponding eigenvalue plot. For example, if there is a large separation between the eigenvalues λ_d and λ_{d+1} , we take the first d eigenvectors as the dominant features of the decomposition, remove the one with zero eigenvalue, and ignore the rest. (The d_1 and d_2 values in Tables 1–4 show the initial d values before we remove the vectors with zero eigenvalues. The embedding and alignment computation are done after we get rid of those vectors.) The initial embedding dimension d usually corresponds to a value around the cluster count in the reference design. Therefore, it is a good idea to start the decomposition with a value a bit larger than the number of functional blocks, then check the plot to find the set of d smallest eigenvalues which is followed by a relatively large λ_{d+1} . However, d does not necessarily have to be equal to the block count. Fewer vectors can divide the space into many partitions, or the eigenvectors may give different granularity for functional clusters of the design than the #FB value. Figure 8 shows example plots for three benchmarks where the gap can be seen clearly for both test and reference circuits.

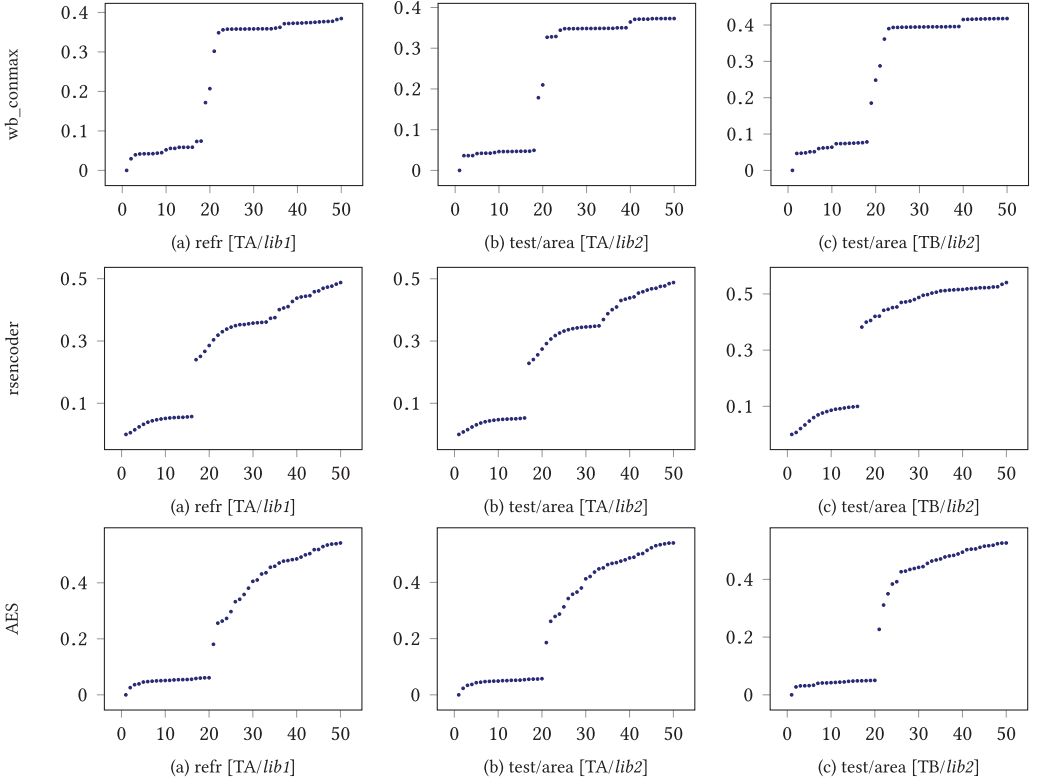


Fig. 8. Eigenvalues for *wb_conmax* (Wishbone conmax IP), *rsencoder* (Reed-Solomon encoder), and *AES* benchmark [refr: time-optimized reference design synthesized with *lib1* using Tool A (TA), test/area: area-optimized test design synthesized with *lib2* using Tool A (TA) and Tool B (TB)].

Overall, both eigendecomposition and I/O-based features prove to be effective embedding tools to find a correspondence between the two circuits and can provide up to 99% accuracy in identifying high-level blocks. The clean-up stage also helps correct the mismatch for both techniques and improves the matching results. Even in cases where there is a considerable size difference between circuits, as can be seen in Tables 1–4 for the Floating-point unit benchmark, for which we could do intense performance optimization without timing violations and could increase area a lot, we get a very high coverage although the reference circuit is almost twice the size of the other. In general, for the initial matching results, I/O-based features provide better results compared to eigenfeatures due to the large number of features (equal to # I/O ports in each benchmark). However, for the Wishbone conmax IP benchmark, we see a different behavior caused by too many features in the I/O vector. Although the circuits in Tables 3 and 4 are not much affected, we observe that when we have larger circuit graphs, the high number of I/O ports can cause some noise as shown in Tables 1 and 2. When matching is done using both I/O and eigendecomposition-based features, eigenvectors help modulate this, and as can be seen from the [M2] columns of Tables 1 and 2, we get higher results in the combined case compared to the results where I/O or eigenfeatures are used alone.

Another interesting result we observe is the difference in the I/O-based matching accuracy of the *sparc* benchmark between the two cases in Tables 1 and 2. Although the circuits for the same benchmark in Tables 3 and 4 have very high matching percentages (see [M2] sections of the I/O

Table 6. Runtime Statistics for Design Benchmarks (Time in Seconds)

Circuits	Time [TA/lib2] (s)			Area [TA/lib2] (s)			Time [TB/lib2] (s)			Area [TB/lib2] (s)		
	I/O-Feat	E-Feat	I/O & E	I/O-Feat	E-Feat	I/O & E	I/O-Feat	E-Feat	I/O & E	I/O-Feat	E-Feat	I/O & E
Wishbone memory contr.	42	30	46	41	30	48	36	39	51	36	43	55
MIPS32 processor	328	768	752	384	1446	1396	430	1825	1864	416	1242	1375
PID controller	5	5	7	5	5	7	7	7	8	7	6	8
Reed-Solomon encoder	10	4	11	9	4	10	9	4	10	10	5	10
8051 microcontroller	872	1154	1347	962	1163	1371	338	714	694	394	720	700
Wishbone conmax IP	4118	477	4006	4043	409	3993	493	1557	4068	3991	1157	3889
Floating-point unit	2405	4750	4404	2454	5678	5459	2465	6273	6261	2915	5109	5312
AES cipher	83	19	92	80	19	88	139	64	144	131	96	141
OpenSparc T1 processor	13387	129183	136771	26807	83534	78785	80567	92961	132576	70646	94910	115032

feature column in Tables 3 and 4), Tables 1 and 2 show that when the test graph is significantly larger than the reference graph, there is a noticeable change in the I/O-based matching results, even after the clean-up/correction stage for unlabeled nodes (see corresponding [M2] columns). The size difference in graphs is an important factor that can affect the I/O-based matching in large circuits. As explained in the previous section, this is also the reason why for designs where one graph is relatively larger than the other, we recommend doing a first round of correction during the clean-up stage even for the already labeled nodes, then continuing the next iterations with unlabeled ones ($\sim 5k$ node gap is a good threshold for this decision). The results show that eigendecomposition is more resilient to size differences than I/O-based feature extraction. The significant improvement between the [M1] and [M2] results for eigendecomposition-based features, we believe, is due to the initial sparsity of the labeled nodes in the test circuit—caused by the large size difference.

In conclusion, for the common cases where the number of I/O features is not extremely large (based on our results, if the number of I/O ports to circuit size ratio is less than 10%) and the size of the test circuit is around the same range as the reference design after removing the buffers in both circuits, our suggestion is using I/O features alone to avoid disturbing the precision with the other set, and using the combination of both I/O and eigendecomposition-based features when the number of I/O ports is unusually large. For the cases where the size difference between the circuit graphs is significant—which usually means to meet the constraints with the given technology library, the circuit was obtained using more hardware and area resources and ended up with a larger area cost compared to the other graph—we again recommend the combination of both features as for designs with large size gaps the number of I/O ports may not be enough to provide an accurate embedding, whereas eigenfeatures are more sound ($\sim 10^4$ gap is a reasonable threshold for this in general).

We believe that the eigendecomposition-based technique is a powerful tool for robustness of our technique together with I/O features and provides accurate embedding coordinates by only using a relatively smaller number of features. Since the results may vary depending on the design and both methods after the clean-up stage provide high enough accuracy for a human analyst to correct the boundaries by manual analysis, our suggestion is to make a decision between the two techniques based on the number of I/O ports to circuit size ratio and the difference in the graph sizes as explained above.

Table 6 shows the runtime statistics for our experiments. The number of features and how quickly the features can distinguish the assignments can change the runtime a lot, but the sizes of the compared circuits and the sparsity values in general are the two major factors that affect the runtime as can be seen by reviewing the circuits sizes in Tables 1–4 in comparison. The speed can be reduced with smaller sparsity values. There might be a slight tradeoff for accuracy, but the method is in general quite robust and accuracy would not drastically change. When the

ρ_s parameter is over a certain value, even if we set it to larger numbers, the assignments do not change significantly. Hence, in our experiments, we preferred to pick numbers around such saturation points to set the value of ρ_s . As said before, the average cluster size in the reference circuit would give a good estimate for this. The code can be optimized for performance in industrial uses with a better optimized code implementation, and the speed can be improved with faster processors and more parallelization (a full parallelization can be utilized for cost matrix computation). We tried our best for academic research demonstration and obtained reasonable runtimes for a decision problem with a high complexity nature.

Another discussion that we would like to raise to the attention of the reader is about the assignment algorithm we employ in the final stage. In our work, we tried to provide a framework for geometric embedding of circuit graphs, presented two methods for a robust feature extraction, and showed the efficacy of our work by using an efficient linear assignment solver. Yet, for a future work that may use the findings of our research, it is worth mentioning that the final step can be replaced by another matching algorithm depending on the needed efficiency-speed tradeoffs. A modified Hungarian algorithm for sparse or dense matrices, parallelized assignment algorithms or integer linear programming (ILP) formulation-based approaches are a few suggestions and are all interesting to check. We would be happy to see the results of such extensions. There are a lot of solutions for the linear assignment problem and we would like to suggest [8] for a comprehensive study of the algorithms to interested readers.

We would like to once again emphasize that the coverage targeted by our tool is not a fine matching like a shift register or counter identification. It is at the block level and uses the reference design information to infer where these blocks are defined in the test circuit, for which how fine (or coarse) the matching we aimed can be seen in our results by checking the #FB (number of functional blocks) parameter in the table. This parameter and the circuit size determine how high-level the coverage is for each design. Our tool addresses the problem of finding block boundaries in a sea of logic gates for an untrusted test circuit. The final matching results in the [M2] column of Tables 1–4 demonstrate the efficacy of our method which can achieve over 90% accuracy in identifying such high-level blocks by only using a relatively small number of features.

4 CONCLUSIONS

With globalized and untrusted IC design and manufacturing processes, reverse engineering is gaining importance for detecting malicious changes. In this article we addressed the problem of functional partitioning for reverse engineering. We presented a methodology to compute a correspondence between the nodes of a reference and a test design using geometric embedding. This helps partition a test design into high-level blocks corresponding to the reference design. This is a critical step towards the end goal of checking the function of each partition to ensure the absence of malicious changes. The identified high-level blocks can be further analyzed for verifying their functionality or identifying malicious changes. The experiments demonstrate the practical efficacy of this partitioning method on eight circuits from OpenCores as well as the OpenSparc T1 processor.

REFERENCES

- [1] [n. d.]. OpenCores 2016. *Digital Open Source Hardware Cores*. Retrieved August 2016 from <http://opencores.org/>.
- [2] [n. d.]. OpenSparc T1 2016. *Open Source Multi-core Processor*. Retrieved August 2016 from <http://oracle.com/technetwork/systems/opensparc/opensparc-t1-page-1444609.html>.
- [3] Charles J. Alpert and Andrew B. Kahng. 1993. Geometric embeddings for faster and better multi-way netlist partitioning. In *Proceedings of the Design Automation Conference (DAC'93)*.
- [4] Charles J. Alpert and Andrew B. Kahng. 1996. Simple eigenvector-based circuit clustering can be effective. In *Proceedings of the International Symposium on Circuits and Systems (ISCAS'96)*.

- [5] Charles J. Alpert, Andrew B. Kahng, and So-Zen Yao. 1999. Spectral partitioning with multiple eigenvectors. *Discr. Appl. Math.* 90, 1-3 (Jan. 1999), 3–26.
- [6] Shawki Areibi and Anthony Vannelli. 1993. A combined eigenvector tabu search approach for circuit partitioning. In *Proceedings of the IEEE Custom Integrated Circuits Conference 1993*. 9.7.1–9.7.4.
- [7] Xiao Bai, Hang Yu, and Edwin R. Hancock. 2004. Graph matching using spectral embedding and alignment. In *Proceedings of the International Conference on Pattern Recognition (ICPR'04)*.
- [8] R. Burkard, M. Dell'Amico, and S. Martello. 2012. *Assignment Problems*. Society for Industrial and Applied Mathematics.
- [9] Burcin Cakir and Sharad Malik. 2015. Hardware trojan detection for gate-level ICs using signal correlation-based clustering. In *Proceedings of the Conference on Design, Automation and Test in Europe (DATE'15)*.
- [10] Pak K. Chan, Martine D. F. Schlag, and Jason Y. Zien. 1994. Spectral K-way ratio-cut partitioning and clustering. *IEEE Trans. Comput.-Aid. Des. Integr. Circ. Syst.* 13, 9 (1994), 1088–1096.
- [11] Pak K. Chan, Martine D. F. Schlag, and Jason Y. Zien. 1995. Spectral-based multi-way FPGA partitioning. In *Proceedings of the 3rd International ACM Symposium on Field-Programmable Gate Arrays*. 133–139.
- [12] Ryan R. Curtin, James R. Cline, Neil P. Slagle, William B. March, P. Ram, Nishant A. Mehta, and Alexander G. Gray. 2013. MLPACK: A scalable C++ machine learning library. *J. Mach. Learn. Res.* 14 (2013), 801–805.
- [13] Jack Edmonds and Richard M. Karp. 1972. Theoretical improvements in algorithmic efficiency for network flow problems. *J. ACM* 19, 2 (Apr. 1972), 248–264.
- [14] Jon Frankle and Richard M. Karp. 1986. Circuit placements and cost bounds by eigenvector decomposition. In *Proceedings of the IEEE International Conference on Computer-Aided Design*. 414–417.
- [15] Adria Gascón, Pramod Subramanyan, Bruno Dutertre, Ashish Tiwari, Dejan Jovanovic, and Sharad Malik. 2014. Template-based circuit understanding. In *Proceedings of the International Conference on Formal Methods in Computer-Aided Design (FMCAD'14)*.
- [16] Gene H. Golub and Charles F. Van Loan. 2013. Large sparse eigenvalue problems. In *Matrix Computations* (4th ed.). The Johns Hopkins University Press, Baltimore, MD.
- [17] Scott W. Hadley, Brian L. Mark, and Anthony Vannelli. 2006. An efficient eigenvector approach for finding netlist partitions. *Trans. Comp.-Aided Des. Integr. Cir. Sys.* 11, 7 (Nov. 2006), 885–892.
- [18] Lars Hagen and Andrew B. Kahng. 1992. A new approach to effective circuit clustering. In *Proceedings of the International Conference on Control, Automation and Diagnosis (ICCAD'92)*.
- [19] Mark C. Hansen, Hakan Yalcin, and John P. Hayes. 1999. Unveiling the ISCAS-85 benchmarks: A case study in reverse engineering. *IEEE Des. Test Comput.* 16, 3 (July 1999), 72–80.
- [20] R. Jonker and A. Volgenant. 1987. A shortest augmenting path algorithm for dense and sparse linear assignment problems. *Computing* 38, 4 (Nov. 1987), 325–340.
- [21] Tomas Kazmar. 2017. Linear Assignment Problem Solver—GitHub repository. Retrieved November 2017 from <https://github.com/gatagat/lap/commit/3d8c4d060d0886cdd708e2895f18a623f1fec638>.
- [22] Erwin Kreyszig. 2010. *Advanced Engineering Mathematics* (10th ed.). John Wiley & Sons, Inc., New York, NY, 339–340.
- [23] Lanczos algorithm. 2017. Lanczos algorithm—Wikipedia, The Free Encyclopedia. Retrieved July 21, 2017 from https://en.wikipedia.org/w/index.php?title=Lanczos_algorithm&oldid=786899578.
- [24] Wenchao Li, Adria Gascon, Pramod Subramanyan, Wei Yang Tan, Ashish Tiwari, Sharad Malik, Natarajan Shankar, and Sanjit A. Seshia. 2013. WordRev: Finding word-level structures in a sea of bit-level gates. In *Proceedings of the (HOST'13)*.
- [25] Wenchao Li, Zach Wasson, and Sanjit A. Seshia. 2012. Reverse engineering circuits using behavioral pattern mining. In *Proceedings of the IEEE International Symposium on Hardware Oriented Security and Trust (HOST'12)*.
- [26] Ulrike Luxburg. 2007. A tutorial on spectral clustering. *Stat. Comput.* 17, 4 (Dec. 2007), 395–416.
- [27] Janett Mohnke, Paul Molitor, and Sharad Malik. 1999. Establishing latch correspondence for sequential circuits using distinguishing signatures. *Integr. VLSI J.* 27, 1 (Jan. 1999), 33–46.
- [28] Andrew Y. Ng, Michael I. Jordan, and Yair Weiss. 2001. On spectral clustering: Analysis and an algorithm. In *Advances in Neural Information Processing Systems*. MIT Press, 849–856.
- [29] Bernhard M. Riess, Konrad Doll, and Frank M. Johannes. 1994. Partitioning very large circuits using analytical placement techniques. In *Proceedings of the 31st Annual Design Automation Conference (DAC'94)*. ACM, New York, NY, 646–651.
- [30] Conrad Sanderson and Ryan Curtin. 2016. Armadillo: A template-based C++ library for linear algebra. *J. Open Source Softw.* 1, 2 (June 2016), 26.
- [31] Blair Schiffrer, Jianhua Li, and Laleh Behjat. 2005. A multivalued eigenvalue based circuit partitioning technique. In *Proceedings of the 5th IEEE International Workshop on System-on-Chip for Real-Time Applications (IWSOC'05)*. 312–316.
- [32] Yiqiong Shi, Chan Wai Ting, Bah-Hwee Gwee, and Ye Ren. 2010. A highly efficient method for extracting FSMs from flattened gate-level netlist. In *Proceedings of the International Symposium on Circuits and Systems (ISCAS'10)*.

- [33] Mathias Soeken, Baruch Sterin, Rolf Drechsler, and Robert Brayton. 2015. Simulation graphs for reverse engineering. In *Proceedings of the International Conference on Formal Methods in Computer-Aided Design (FMCAD'15)*.
- [34] Pramod Subramanyan, Nestan Tsiskaridze, Wenchao Li, Adria Gascón, Wei Yang Tan, Ashish Tiwari, Natarajan Shankar, Sanjit A. Seshia, and Sharad Malik. 2014. Reverse engineering digital circuits using structural and functional analyses. *IEEE Trans. Emerg. Top. Comput.* 2, 1 (2014), 63–80.
- [35] A. Volgenant. 1996. Linear and semi-assignment problems: A core oriented approach. *Comput. Operat. Res.* 23, 10 (Oct. 1996), 917-932.

Received May 2017; revised February 2018; accepted March 2018